

دليل شرح كامل — TASK 01: File Organizer (AVIP 2026)

هذا الملف هو الشرح الكامل المرافق للمشروع، ويُقرأ بالتوازي مع الكود الموجود في مجلد `/src` و `/tests`. الكود في هذا الدليل مطابق لما تم تسليمه فعليًا في المشروع.

ملاحظة صدق مهمة قبل أي شيء (مطلوبة في PART 15 من طلبك)

بيئة التنفيذ التي أعمل بداخلها الآن لا تحتوي على .NET SDK مثبت (تحقق فعليًا بتشغيل `dotnet --version` وكانت النتيجة "not found"). هذا يعني:

- لم أقم فعليًا بتشغيل `dotnet build` أو `dotnet test` على هذا الكود.
- كل ما أكتبه أدناه عن "يعمل بشكل صحيح" هو تحقق منطقي (logical verification) قرأت فيه كل سطر، تتبعت التوافق بين التوقيعات (signatures)، تأكدت من الـ namespaces والـ using، وتتبع تدفق البيانات بين الكلاسات — وليس تنفيذًا فعليًا.
- أنت مطالب بتشغيل `dotnet build` و `dotnet test` بنفسك على جهازك (الخطوات في PART 8) والتأكد من عدم وجود أخطاء قبل الدفع إلى GitHub. هذا جزء من عملك كمتدرب، وليس شيئًا يجب أن أدعي أنني قمت به نيابة عنك.
- راجعت الكود أكثر من مرة بحثًا عن: أخطاء تسمية، تعارض توقيعات، استخدام namespaces غير مستوردة، nullable warnings، ومنطق غير متسق — ولم أجد شيئًا يمنع الترجمة (compilation) حسب معرفتي بـ C#/.NET، لكن هذا يبقى تقييمًا منطقيًا وليس ضمانًا مطلقًا.

شرح المهمة بالكامل من الصفر — PART 1

1. ما هو File Organizer؟

برنامج صغير يفحص مجلدًا معينًا (Source) ويقوم بنقل كل ملف بداخله إلى مجلد فرعي آخر (داخل Target) بناءً على نوع الملف (صورة، مستند، أرشيف، صوت، فيديو...). الفكرة تشبه تمامًا ما يفعله "الترتيب التلقائي" في بعض تطبيقات إدارة الملفات، لكن هنا نبنيه نحن من الصفر بلغة C#.

2. ما المشكلة التي يحلها؟

مجلد التنزيلات (Downloads) عادةً يمتلئ بخليط عشوائي من الملفات: صور، PDF، ملفات مضغوطة، مقاطع صوت وفيديو... والترتيب اليدوي:

- بطيء وممل.
- عرضة للخطأ البشري (حذف/استبدال ملف بالخطأ).
- غير قابل للتكرار كل يوم بنفس الطريقة.

الأتمتة (Automation) تحل هذه المشكلة: قاعدة ثابتة تُطبّق بلا كلل ولا أخطاء.

3. ما الذي يتوقعه التدريب بالضبط من TASK 01؟

بناء تطبيق Console بلغة C#/.NET يقوم:

- بمجلد Source.
- بنقل ملف حسب امتداده.
- بنقله إلى مجلد فرعي مناسب داخل Target.
- عدم الكتابة فوق أي ملف موجود مسبقًا (No Overwrite).

- حل التعارضات بأسماء الملفات بطريقة "حتمية" (Deterministic) — أي أن نفس المدخلات تعطي دائمًا نفس النتيجة.
- قبول مسار Source و Target من المستخدم (CLI أو ملف إعدادات)، وليس مكتوبًا بشكل ثابت (hard-coded) في الكود.
- توفير وضع "تجربة بدون تنفيذ" (Dry-Run) يُظهر ما سيحدث دون أن يحدث فعليًا.

4. كل متطلب رسمي (Official Requirement) — ملخص مرقّم

هذه هي المتطلبات كما وردت في ملف AVIP الرسمي، ولم يتم إسقاط أو تخفيف أي منها:

1. فرز الملفات داخل مجلد إلى مجلدات فرعية حسب النوع (صور/مستندات/أرشيف/صوت/فيديو).
2. التعامل مع الامتدادات الشائعة ووضعها في المجلد الصحيح.
3. عدم الكتابة فوق ملف موجود.
4. حل تعارض الأسماء بشكل حتمي.
5. قبول مسار مصدر وهدف قابلين للتهيئة (CLI أو ملف إعدادات).
6. خيار Dry-Run/Logging يعرض العمليات المخطط لها دون تنفيذها.
7. التسليمات: مستودع README، GitHub، بأمثلة استخدام، سجل تشغيل نموذجي (sample run log)، لقطات شاشة قبل/بعد.

كل بند من هذه مُنفذ فعليًا في المشروع (الجدول الكامل موجود في `README.md` تحت "Internship Requirements Checklist").

5. ما هو مُدخل (Input) البرنامج؟

- مسار مجلد المصدر (Source Directory) — نصّ (string) يمثل مسارًا على القرص.
- مسار مجلد الهدف (Target Directory) — نفس الشيء.
- أعلام اختيارية: `--log <path>`، `--recursive`، `<dry-run>`.
- بديل: ملف إعدادات نصي بسيط بصيغة `key=value`.

6. ما هو مُخرَج (Output) البرنامج؟

- ملفات مُنقولة فعليًا (في الوضع العادي) إلى `<Target/<Category>/<filename>`.
- نص في الطرفية (Console) يوضح كل عملية، بالإضافة إلى ملخص نهائي.
- ملف سجل اختياري على القرص إذا استخدم `log--`.
- `exit code` يعكس نجاح التشغيل أو فشله (مفيد عند استخدام البرنامج داخل سكريبتات آلية).

7. السلوك المتوقع الكامل

1. قراءة المدخلات والتحقق منها.
2. التأكد من وجود مجلد Source.
3. التأكد من أن Target ليس نفس Source وليس متداخلًا بداخله.
4. تعداد (enumerate) كل الملفات المباشرة داخل Source (بدون الدخول إلى المجلدات الفرعية، إلا إذا استخدم `--recursive`).
5. لكل ملف: استخراج الامتداد → تصنيفه → حساب المسار الوجهة → التحقق من وجود تعارض → حل التعارض إن وُجد.
6. في الوضع العادي: إنشاء المجلد الوجهة إن لم يكن موجودًا، ثم نقل الملف.
7. في وضع Dry-Run: لا يتم إنشاء أي مجلد ولا نقل أي ملف، فقط طباعة/تسجيل ما كان سيحدث.

8. طباعة ملخص نهائي (عدد الملفات المفحوصة، المنظّمة، الأخطاء).

8. فئات الملفات (Categories) التي سندعمها

(فئة احتياطية شاملة) `Others`، و `Video`، `Audio`، `Archives`، `Documents`، `Images`.

9. الامتدادات الشائعة لكل فئة

موجودة بالتفصيل في `FileClassifier.cs` وفي جدول `README.md`. باختصار:

- **Images:** jpg, jpeg, png, gif, bmp, webp, svg, tiff/tif, heic.
- **Documents:** pdf, doc/docx, txt, rtf, xls/xlsx, csv, ppt/pptx, odt, md.
- **Archives:** zip, rar, 7z, tar, gz, bz2, xz.
- **Audio:** mp3, wav, flac, aac, ogg, m4a, wma.
- **Video:** mp4, mkv, avi, mov, wmv, webm, flv, m4v.

10. ماذا يحدث للامتدادات غير المدعومة؟

تذهب تلقائيًا إلى فئة `Others`. لم أختَر رفضها أو تجاهلها، لأن هدف الأداة هو عدم فقدان أي ملف أبدًا — كل ملف يجب أن ينتهي في مكان ما يمكن العثور عليه.

11. ماذا يحدث للملفات بلا امتداد؟

نتحقق من ذلك `FileClassifier.Classify` يُعيد نصًا فارغًا `""`. في `Path.GetExtension("README")` أيضًا، بدل أن يُرمى استثناء أو يُتجاهل `Others` ونضع الملف في `(string.IsNullOrEmpty(extension))` (صراحة الملف).

12. ماذا يحدث عندما يكون اسم الملف الوجهة موجودًا بالفعل؟

لا نكتب فوقه أبدًا. نستدعي `ConflictResolver.Resolve(...)` الذي يبحث عن أول اسم بديل غير مستخدم بإضافة `1_`, `2_`, `3_` ... إلخ قبل الامتداد الأخير فقط.

13. كيف يعمل حل التعارض الحتمي بالتفصيل؟

- الخطوة صفر: جرّب الاسم الأصلي كما هو.
- إن كان موجودًا: افصل الاسم إلى (بدون الامتداد) + (الامتداد الأخير) باستخدام `Path.GetFileNameWithoutExtension` و `Path.GetExtension`.
- أضف `1_` إلى الجزء الأول، أعد التركيب، تحقق مرة أخرى.
- استمر في الزيادة (`2_`, `3_`, ...) حتى تجد اسمًا غير مستخدم.
- النتيجة حتمية: لنفس حالة القرص، ستحصل دائمًا على نفس اسم الملف الناتج — لا عشوائية ولا اعتماد على الوقت (لم نستخدم Timestamp أو GUID لأن ذلك غير حتمي بالمعنى الذي يريده التدريب، وأيضًا أقل قابلية للقراءة من قبل الإنسان).

14. ما معنى وضع Dry-Run؟

تنفيذ "جاف" — البرنامج يقوم بكل خطوات القرار (الفحص، التصنيف، حساب الوجهة، حل التعارض) لكنه لا يلمس القرص فعليًا: لا `Directory.CreateDirectory`، ولا `File.Move`. الفائدة: تستطيع معاينة النتيجة قبل أن تثق بالبرنامج بملفاتك الحقيقية.

15. ما معنى Logging هنا؟

تسجيل نصي منظم لكل خطوة مهمة: بداية التشغيل، المسارات، الوضع (Dry-Run أم لا)، كل عملية نقل/تخطيط، الأخطاء، والملخص النهائي. يُطبع على الطرفية دائماً، ويُكتب أيضاً إلى ملف إذا مرّرت `--log`.

16. لماذا الـ Logging مفيد؟

- إثبات (Audit trail) لما حدث فعلياً، خصوصاً عند نقل مئات الملفات.
- تشخيص الأخطاء دون الحاجة لإعادة التشغيل.
- أحد متطلبات التسليم الرسمية للتدريب (sample run log).

17. لماذا يجب أن يكون مساري Source و Target قابلين للتهيئة؟

لو كانا مكتوبين بشكل ثابت (hard-coded) في الكود:

- البرنامج يعمل فقط على جهاز مطوّره.
- كل تغيير في المسار يتطلب إعادة بناء (rebuild) البرنامج.
- غير قابل لإعادة الاستخدام أو الأتمتة داخل سكريبتات أخرى. التهيئة عبر CLI/ملف إعدادات تجعل الأداة عامة وقابلة لإعادة الاستخدام — وهذا أيضاً شرط رسمي صريح في التدريب.

18. ماذا يحدث إن لم يكن مجلد المصدر موجوداً؟

وإن لم يكن موجوداً يرمي `Directory.Exists` يتحقق أولاً بـ `FileOrganizerService.Organize` يلتقط هذا الاستثناء تحديداً ويُطبع رسالة نظيفة `Program.cs`. برسالة واضحة `DirectoryNotFoundException` exit code = 2. ويُنتهي البرنامج بـ (مخيف Stack Trace بدون)

19. ماذا يحدث إن لم يكن مجلد الهدف موجوداً؟

يُنشأ تلقائياً (`Directory.CreateDirectory`) — لكن فقط في الوضع العادي (ليس Dry-Run)، لأن إنشاء مجلد هو بحد ذاته تغيير فعلي على القرص، ويجب ألا يحدث في وضع المعاينة.

20. ماذا يحدث إن تعذّر الوصول إلى ملف؟

نلتقط `UnauthorizedAccessException` / `IOException` / `PathTooLongException` حول عملية النقل الخاصة بذلك الملف فقط (وليس حول التشغيل بأكمله)، ونسجّل تلك الحالة كـ `Error` في النتائج، ونُكمل معالجة بقية الملفات. فشل ملف واحد لا يوقف التشغيل بأكمله.

21. ماذا يحدث إن تعذّر نقل ملف؟

نفس المعالجة أعلاه — يُسجّل كخطأ في السجل والملخص النهائي (`Errors: N`)، ويُرجع البرنامج exit code = 2 إن كان هناك خطأ واحد على الأقل.

22. كيف يتصرف التطبيق بأمان بشكل عام؟

- لا يكتب فوق أي ملف (`overwrite: false` صراحة في `File.Move`).
- لا يقوم بأي عملية حذف على الإطلاق.
- يرفض العمل إذا كان Target هو نفسه Source أو متداخلاً بداخله (لتفادي إعادة معالجة نفس الملفات في حلقة).
- Dry-Run لا يستدعي أي API يُغيّر حالة القرص إطلاقاً.

23. لماذا `System.IO` مناسبة هنا؟

هي مساحة الأسماء (namespace) القياسية في .NET للتعامل مع نظام الملفات: قراءة/كتابة/نقل/فحص وجود الملفات والمجلدات. لا حاجة لأي مكتبة خارجية للقيام بهذه العمليات الأساسية.

24. لماذا Path و FileInfo / DirectoryInfo مفيدة؟

- للتعامل مع نصوص المسارات (دمج، استخراج الامتداد، استخراج الاسم بدون الامتداد...) دون (static) دوال ثابتة: **Path**. الحاجة لملف حقيقي على القرص.
- بدلاً من إنشاء **Path** و **File** و **Directory** في هذا المشروع اخترنا الاعتماد بشكل أساسي على الدوال الثابتة في الكاملة، لأن حاجتنا هنا بسيطة (تعداد، فحص وجود، نقل) ولا نحتاج لتتبع حالة **DirectoryInfo** / **FileInfo** كائنات (see PART 2). وهذا قرار تبسيط واعٍ

25. أهم كلاسات/APIs التي سنستخدمها

Directory.EnumerateFiles, **Directory.Exists**, **Directory.CreateDirectory**,
File.Exists, **File.Move**, **File.ReadAllLines/WriteAllText**, **Path.GetExtension**,
Path.GetFileName, **Path.GetFileNameWithoutExtension**, **Path.Combine**,
Path.GetFullPath, **Path.TrimEndingDirectorySeparator**.

26. البنية العامة (Architecture) — نظرة سريعة

طبقات صغيرة كل واحدة بمسؤولية واحدة (Single Responsibility): **Program** (تشغيل) → **CommandLineOptions** (تحويل args إلى Options) → **FileOrganizerService** (تنسيق العملية) يستخدم **FileClassifier** (قرار: أي فئة؟) و **ConflictResolver** (قرار: أي اسم نهائي؟) ويكتب عبر **OrganizerLogger** (عرض/تسجيل). تفاصيل أوسع في PART 2.

27. تدفق التنفيذ الكامل (ملخص سريع، والتفاصيل في PART 10)

Main → **args** تحليل → **OrganizerLogger** و **FileOrganizerService** → **service.Organize(...)** (تحقق من المسارات → تعداد الملفات → لكل ملف: تصنيف → حل تعارض → تنفيذ/تخطيط) → **exit code** طباعة الملخص → إرجاع

PART 2 (Architecture) التقنية والبنية المعمارية —

اختيار إصدار .NET

الإصدار الموصى به: .NET 10 (LTS)، مع C# 14.

السبب (تم التحقق منه عبر بحث ويب لأن معلومات إصدارات .NET تتغير مع الوقت):

- كلاهما ينتهي دعمهما الرسمي في 10 نوفمبر 2026 — أي بعد شهرين فقط من كتابة هذا الدليل (نحن 9.NET و 8.NET. الآن في سبتمبر 2026). بدء مشروع تدريبي جديد بإصدار على وشك الانتهاء دعمه قرار غير عملي.
- صدر في نوفمبر 2025 ومدعوم حتى نوفمبر 2028 — أي ثلاث سنوات دعم (دعم طويل الأمد) LTS إصدار 10.NET. كاملة من لحظة كتابة هذا المشروع تقريباً.
- بثلاث سنوات دعم، وكل إصدار فردي (9, 11...) هو LTS يتبع نمطاً ثابتاً: كل إصدار زوجي (8, 10, 12...) هو .NET. هو الخيار LTS، بسنتين دعم فقط. لمشروع تعليمي/بورتفوليو تريد أن يبقى قابلاً للتشغيل والتحديث لأطول فترة ممكنة STS الصحيح دائماً.
- لأي سبب، بإمكانك تغيير سطر 10.NET. ولا يمكنك تثبيت 8.NET SDK. إذا كان جهازك يحتوي فقط على الكود net8.0 إلى csproj. في كلا ملفي <TargetFramework>net10.0</TargetFramework> و 10.NET. سيعمل بلا أي تغيير آخر — لم نستخدم أي ميزة حصرية على

فلسفة البنية: لماذا هذا الشكل بالتحديد؟

طبّقنا نفس الشكل المقترح في طلبك تقريبًا حرفيًا، لأنه بالفعل بنية معقولة واحترافية لمشروع بهذا الحجم:

```
src/FileOrganizer/
├─ Program.cs                # نقطة الدخول فقط (Composition Root)
├─ Models/                  # بيانات بلا سلوك (records/enums)
│   ├── FileCategory.cs
│   ├── OrganizerOptions.cs
│   └─ FileOperationResult.cs
├─ Services/                # المنطق: قرارات + تنفيذ
│   ├── FileOrganizerService.cs # المنسق (orchestrator)
│   ├── FileClassifier.cs      # قرار: أي فئة؟ (pure/stateless)
│   └─ ConflictResolver.cs     # قرار: أي اسم نهائي؟ (pure/testable)
├─ Configuration/
│   └─ CommandLineOptions.cs   # args/config file → Options
└─ Logging/
    └─ OrganizerLogger.cs      # عرض/تسجيل فقط
```

لماذا هذا الفصل وليس أكثر (ولا أقل)؟

- لأنهما "قرار" **FileOrganizerService** منفصلان عن **ConflictResolver** و **FileClassifier** وحتى هذا، **ConflictResolver** باستثناء فحص وجود ملف (في) لا يلمسان القرص فعليًا (pure decision) "خالص هذا يسمح باختبارهما بمئات الحالات دون إنشاء أي. (**Func<string, bool>**) قابل للحقن/الاستبدال في الاختبارات عبر ملف حقيقي على القرص — اختبارات أسرع وأوضح.
- **FileOrganizerService** هو الوحيد الذي يُنفَّذ فعليًا (**Directory.CreateDirectory**، **File.Move**)، لكن دون تعقيد إضافي — CQRS يُشبه فكرة) فصل "القرار" عن "التنفيذ" مبدأ معماري معروف. (لهذا framework لم نستورد أي).
- **Program.cs** منفصل عن **CommandLineOptions** حتى يمكن اختبار منطق تحليل الأعلام (**--dry-run**، **-**، **-source**...) دون تشغيل عملية (process) سطر أوامر حقيقية بمعاملات سطر أوامر.
- **Microsoft.Extensions.Logging** بسيط جدًا عمدًا — لا نستخدم **OrganizerLogger** (وهي مكتبة قوية) لأداة صغيرة. (معقّد (DI) حقن اعتمادية، (levels) مستويات، (providers) لكن مصممة لتطبيقات بها عدة مزودي سجل قرار متعمّد للحفاظ على البساطة — (over-engineering) "ذات غرض واحد، هذا "أكثر من اللازم).

ما الذي لم نفعله عمدًا (تجنبًا للمبالغة الهندسية):

- لا واجهات (interfaces) لكل كلاس — لا حاجة فعلية لها هنا لأننا لا نملك أكثر من تنفيذ واحد لكل خدمة، ولا نستخدم حاوية DI حقيقية.
- لا مكتبة حقن اعتمادية (Dependency Injection container) — الكلاسات القليلة تُبنى يدويًا بسطرين في (**new FileClassifier()**، **new ConflictResolver()**...) **Program.cs**، وهذا أوضح لمتعلّم من إعداد **IServiceCollection** لثلاث أو أربع خدمات فقط.
- لا Design Patterns معقّدة (لا Factory، لا Strategy منفصلة، لا Repository) — التصنيف عبارة عن قاموس (Dictionary) واحد، وهذا يكفي تمامًا ولا يستدعي نمط تصميم كامل.
- لا معالجة غير متزامنة (**async/await**) — عمليات نظام الملفات هنا صغيرة الحجم ومتزامنة بطبيعتها في .NET. (**File.Move**) ليست **async** أصلًا، فإدخال **async** كان سيُضيف تعقيدًا بلا فائدة حقيقية.

تصميم الـ CLI

اخترنا صياغة أعلام صريحة (`source`, `--target`, `--dry-run`, `--recursive`, `--log`, `-- --`) بدلاً من معاملات موضعية (positional arguments) لأن:

- الترتيب لا يهم للمستخدم (`target X --source Y--`) يعمل تمامًا مثل العكس).
- إضافة علم جديد مستقبلاً لا يكسر الاستخدام الحالي.
- رسائل الخطأ أوضح ("Missing required option: --source").

لم نستخدم حزمة `System.CommandLine` الرسمية من مايكروسوفت رغم قوتها، لأن حجم هذا المشروع (٦ أعلام فقط) لا يبرر إضافة تبعية خارجية جديدة — وهذا يتماشى مع قاعدة "تجنب التبعية غير الضرورية" في متطلباتك.

PART 3 — الكود الكامل

كل ملفات الكود المصدري (`Program.cs` , ونماذج `/Models` , وخدمات `/Services` , وإعدادات `/Configuration` , وتسجيل `/Logging` , وكل ملفات الاختبار في `/tests/FileOrganizer.Tests`) مكتملة بالفعل ومُسلّمة كمفاتيح حقيقية قابلة للترجمة ضمن هذا التسليم — لا يوجد أي "TODO" أو "implement this here" في أي مكان. الشرح التفصيلي لكل سطر موجود في PART 9 أدناه، وتصميم الـ CLI مشروح في نهاية PART 2. لتفادي تكرار نفس الكود مرتين في هذا الملف، راجع الملفات الفعلية مباشرة أثناء قراءة PART 9 — كل مقتطف هناك مطابق حرفياً لما هو موجود على القرص.

PART 9 — شرح الكود سطرًا بسطر (كل الملفات)

بما أن الطالب يقصد التعلّم لا فقط النسخ، الشرح هنا مبني على "مجموعات أسطر مترابطة منطقيًا" كما سمح طلبك صراحة ("explain every line or small logically connected group of lines")، بدل تفكيك كل سطر مفرد بلا داعٍ (مثل شرح { } و { } بمفردهما). كل مفهوم مستخدم فعليًا في الكود مشروح، ولا شيء غير مستخدم تم شرحه.

1) `Models/FileCategory.cs`

```
csharp

namespace FileOrganizer.Models;

public enum FileCategory
{
    Images,
    Documents,
    Archives,
    Audio,
    Video,
    Others
}
```

- `namespace FileOrganizer.Models;` — صياغة "file-scoped namespace" (ميزة من C#) تُغني عن كتابة { } وتغليف الملف بأكمله بمسافة بادئة. مساحة الاسم هنا تجمع كل "النماذج" (بيانات بلا سلوك) في (10+). مكان واحد منطقي.
- `public enum FileCategory { ... }` — `enum` (تعداد) هو نوع بيانات يمثل مجموعة قيم ثابتة معروفة (تعداد): للفتنة لأن `string` مسبقًا. اخترناه بدل

- المُصرِّف (compiler) يمنعك من كتابة قيمة خاطئة إملائيًا ("Imagess" بدل "Images") لن يمر أبدًا مع (enum).
- switch على enum يُنبِّهك إن نسيت حالة.
- كل قيمة (Documents , Images , ...) تُطابق تلقائيًا اسم مجلد الفئة عند استدعاء category.ToString () في الخدمة الرئيسية — استخدام بسيط وفَعَال لتحويل enum إلى نص.
- لضمان أن كل ملف ينتهي في مكان محدد دائمًا (catch-all) فئة احتياطية شاملة — Others

2) Models/OrganizerOptions.cs

csharp

```
public sealed record OrganizerOptions
{
    public required string SourceDirectory { get; init; }
    public required string TargetDirectory { get; init; }
    public bool DryRun { get; init; }
    public bool Recursive { get; init; }
    public string? LogFilePath { get; init; }
}
```

- "مصمم لتمثيل" قيمة غير قابلة للتغيير ، C# 9+ نوع بيانات مُقدَّم من record — public sealed record مقروء يعرض كل ToString() بدل المساواة بالمرجع، و Equality) يعطينا تلقائيًا: مساواة بالقيمة. (immutable) تمنع أي كلاس آخر sealed. وفي رسائل فشل الاختبارات (debugging) الخصائص — مفيد جدًا عند تصحيح الأخطاء من الوراثة منه، لأننا لا نحتاج توسعة هذا النوع.
- public required string SourceDirectory { get; init; } —
 - required (C# 11+) يجبر أي كود يُنشئ هذا الكائن على تمرير قيمة لهذه الخاصية، وإلا فشلت الترجمة: (compile error) null. وليس خطأ وقت التشغيل. هذا أفضل من التحقق اليدوي لاحقًا من أن القيمة ليست (init) وهذا بضمن أن، (القابلة للتغيير لاحقًا set بعكس) الخاصية تُضبط مرة واحدة فقط عند الإنشاء: OrganizerOptions لا يتغير بعد بنائه — أمان إضافي عند تمريره بين عدة طبقات من الكود
- public bool DryRun { get; init; } — false قيمة افتراضية (كل bool في C# تبدأ بـ false). (ما لم تُحدَّد false).
- public string? LogFilePath { get; init; } — علامة ? بعد string يعني أن القيمة يُمكن string <Nullable>enable</Nullable> مفعلة عبر (Nullable Reference Type) null أن تكون في ملف قبل استخدامها null هذا يُجبرنا في بقية الكود على التحقق من (.csproj string.IsNullOrEmpty(options.LogFilePath)) فيمنع المُصرِّف ظهور (NullReferenceException) المفاجئة وقت التشغيل

3) Models/FileOperationResult.cs

csharp

```

public enum OperationStatus { Planned, Moved, Skipped, Error }

public sealed record FileOperationResult
{
    public required string SourcePath { get; init; }
    public required string DestinationPath { get; init; }
    public required FileCategory Category { get; init; }
    public required OperationStatus Status { get; init; }
    public bool ConflictResolved { get; init; }
    public string? ErrorMessage { get; init; }
}

```

- نفس أفكار `record` / `required` / `init` أعلاه، مُطبَّقة هنا على "نتيجة معالجة ملف واحد". هذا الكائن هو "الغة مشتركة" بين `FileOrganizerService` (الذي يُنتجه) و `OrganizerLogger` (الذي يعرضه) و `Program.cs` (الذي يحسب منه الملخص النهائي) — بدل تمرير عدة متغيرات منفصلة بين هذه الطبقات، نُمرر كائنًا واحدًا واضح المعنى.
- `OperationStatus.Skipped` مُعرَّف لكنه غير مُستخدَم فعليًا حاليًا (كل حالة فشل حاليًا تُصنَّف `Error`) — تركناه كنقطة توسعة مستقبلية واضحة (مثلاً: تجاهل الملفات المخفية) بدل حذفه، لأن التعداد لا يُكَلَّف شيئًا ويوثق النية.

4) `Services/FileClassifier.cs` (أهم كلاس تعليميًا)

```

csharp

public sealed class FileClassifier
{
    private static readonly Dictionary<string, FileCategory> ExtensionMap = Bu

    public FileCategory Classify(string fileName)
    {
        var extension = Path.GetExtension(fileName);
        if (string.IsNullOrEmpty(extension))
        {
            return FileCategory.Others;
        }
        return ExtensionMap.TryGetValue(extension.ToLowerInvariant(), out var
            ? category
            : FileCategory.Others;
    }
    ...
}

```

- `private static readonly Dictionary<string, FileCategory> ExtensionMap = BuildExtensionMap();`
 - `private` لا يُستخدم إلا داخل هذا الكلاس:

- `static`: `Classify` يوجد نسخة واحدة فقط لهذا القاموس مشتركة بين كل استدعاءات: `FileClassifier` — منطقي لأن التصنيف ثابت ولا يتغير بين الاستدعاءات — كائن
- `readonly`: القاموس: (هنا عند تحميل الكلاس لأول مرة) ولا يمكن إعادة تعيينها لاحقاً. القاموس `readonly` (تحمي المرجع لا المحتوى) نفسه قابل للتعديل تقنياً لكننا لا نغيره أبداً بعد، (نحامي المرجع لا المحتوى) `BuildExtensionMap()`.
- `Dictionary<string, FileCategory>`: هيكل بيانات "خريطة مفتاح → قيمة" بزم من بحث تقريباً ثابت: `if/else if`. مثالي هنا لأن لدينا عشرات الامتدادات ونحتاج بحثاً سريعاً بدل سلسلة طويلة من، $O(1)$.
- `public FileCategory Classify(string fileName)` — الدالة العامة الوحيدة في هذا الكلاس، تأخذ `FileCategory` اسم ملف (كنص) وتعيد
- `var extension = Path.GetExtension(fileName);` — `Path.GetExtension` دالة من `System.IO` (مثال: `"report.pdf" → ".pdf"`). تُعيد كل ما بعد آخر نقطة في الاسم، متضمنةً النقطة نفسها `""`. لم توجد نقطة أصلاً، تُعيد نصاً فارغاً.
- `if (string.IsNullOrEmpty(extension)) return FileCategory.Others;` — يُغطي `README` (مثل) "حالة" ملف بلا امتداد.
- `ExtensionMap.TryGetValue(extension.ToLowerInvariant(), out var category)` —
 - `.ToLowerInvariant()`: يحوّل الامتداد لحروف صغيرة بغض النظر عن إعدادات اللغة في نظام التشغيل: `.ToLower()` (بعكس) يُعاملان بنفس `.jpg` و `.JPG`. هذا ما يجعل `Culture` العادية التي قد تتأثر بالـ (case-insensitive) الطريقة (وهو أحد المتطلبات الصريحة في الاختبارات المطلوبة).
 - `TryGetValue(key, out value)`: `bool` يُعيد `NET`. نمط قياسي في: `ContainsKey` (إن وُجد، دون الحاجة لاستدعاءين منفصلين `out var category` القيمة في `[key]` وبالتالي أداء أفضل وكود أنظف).
 - `? category : FileCategory.Others` (Conditional/Ternary Operator) عامل الشرط الثلاثي — `Others` إن وُجد أعد الفئة، وإلا أعد:
- `BuildExtensionMap()` تبني القاموس مرة واحدة باستخدام مصفوفات نصوص لكل فئة `(string[] images = [".jpg", ...])` — صياغة "Collection Expression" الجديدة في C# 12+ (`[...] {...}`) لإيجاز الكود.
- `Register(map, images, FileCategory.Images);` — دالة مساعدة خاصة `(private static void Register)` تُكرّر استدعاؤها لكل فئة بدل تكرار حلقة `foreach` خمس مرات — تطبيق مبدأ "لا `foreach` تُكرّر استدعاؤها لكل فئة بدل تكرار حلقة (DRY)."

5) Services/ConflictResolver.cs

csharp

```

public sealed class ConflictResolver
{
    public Func<string, bool> PathExists { get; }
    public ConflictResolver() : this(File.Exists) { }
    public ConflictResolver(Func<string, bool> pathExists) { PathExists = pathExists; }

    public (string ResolvedPath, bool ConflictResolved) Resolve(string desiredFullPath)
    {
        if (!PathExists(desiredFullPath)) return (desiredFullPath, false);

        var directory = Path.GetDirectoryName(desiredFullPath) ?? string.Empty;
        var nameWithoutExtension = Path.GetFileNameWithoutExtension(desiredFullPath);
        var extension = Path.GetExtension(desiredFullPath);

        var attempt = 1;
        string candidate;
        do
        {
            var candidateFileName = $"{nameWithoutExtension}_{attempt}{extension}";
            candidate = Path.Combine(directory, candidateFileName);
            attempt++;
        }
        while (!PathExists(candidate));

        return (candidate, true);
    }
}

```

- `public Func<string, bool> PathExists { get; }` — `Func<string, bool>` هو مباشرة داخل الكلاس، نستقبلها `File.Exists` بدل استدعاء `bool` يمثل "دالة تأخذ نصاً وتُعيد (delegate) مفوض" (يدوي بسيط dependency injection/اعتمادية مُحقنة) كوسيط في الذاكرة بدل الذهاب فعلياً للقرص، `HashSet` نمرر دالة وهمية تتحقق من (`ConflictResolverTests.cs`) فتصبح الاختبارات فورية وموثوقة 100% دون الحاجة لإنشاء ملفات حقيقية.
- `( : كسلوك افتراضي حقيقي File.Exists يستخدم ConflictResolver() الباني بلا وسائط في الإنتاج الفعلي Program.cs هذا ما يستخدمه — (يستدعي الباني الآخر this(File.Exists) في الباني بلا وسائط`
- `(string ResolvedPath, bool ConflictResolved) Resolve(...)` — **tuple** مُسمى **(named tuple)** record/يُعيد قيمتين معاً (المسار النهائي، وهل حدث تعارض أصلاً) دون الحاجة لتعريف كلاس `var (resolvedDestination, conflictResolved) = _conflictResolver.Resolve(...)` منفصل لهذا الغرض البسيط. يُستخدم لاحقاً هكذا.
- `Path.GetDirectoryName(desiredFullPath) ?? string.Empty` — `??` (null-coalescing): `null` نستخدم نصاً فارغاً بدلاً من ذلك لتفادي، (حالة نادرة لمسار بلا مجلد أب) `null` إن أعادت الدالة: لاحقاً.

- هذا هو قلب الحل الصحيح — `Path.GetFileNameWithoutExtension` و `Path.GetExtension` — لمشكلة "النقاط المتعددة": كلا الدالتين تعتمدان على "آخر نقطة فقط" في الاسم، وليس أول نقطة. لذلك مع `"project.final.report.pdf"`:
 - كل شيء عدا آخر) `"project.final.report"` تُعيد `GetFileNameWithoutExtension` (امتداد).
 - فقط `".pdf"` تُعيد `GetExtension`.
 - تمامًا كما يطلب المتطلب الرسمي، — `"project.final.report_1.pdf"`: النتيجة النهائية بعد الدمج الخاطئ `"project_1.final.report.pdf"` وليس.
- تُنفذ مرة واحدة على الأقل) `do-while` حلقة — `do { ... } while (PathExists(candidate));` مناسبة هنا لأننا نعلم مسبقًا أن أول محاولة `_1` يجب أن تُختبر قبل أي قرار (قبل فحص الشرط).
- طريقة: **String Interpolation** — `"${nameWithoutExtension}_{attempt}{extension}"` أو `+`، أوضح للقراءة `string.Concat` حديثة لدمج نصوص ومتغيرات بدل.

6) Configuration/CommandLineOptions.cs

الأفكار الأساسية هنا:

- بمقارنة غير حساسة لحالة الأحرف `Dictionary<string, string?> values` أو ملف الإعدادات في مكان واحد قبل بناء CLI يجمع كل قيم (`StringComparer.OrdinalIgnoreCase`) النهائي `OrganizerOptions`.
- يرفض أي علم غير معروف برسالة خطأ `default` مع حالات صريحة لكل علم معروف، و `switch (token)` واضحة بدل تجاهله بصمت (سلوك أكثر أمانًا للمستخدم).
- تُستخدم لإزالة `--` (تعني: كل الأحرف من الفهرس 2 حتى النهاية) `C#` الحديثة في `Range` صياغة — `token[2..]` (`source`, `target` ...).
- يضيف المفتاح فقط إن لم يكن موجودًا بالفعل — هذا `values.TryAdd(key, value)`: منطق دمج ملف الإعدادات يجعل أعلام سطر الأوامر تتفوق دائمًا على ملف الإعدادات عند التعارض (أولوية منطقية: ما يكتبه المستخدم مباشرة الآن أهم مما هو محفوظ في ملف).
- واحد، وتسمح `bool` إلى (نص فارغ، `"1"`، `"true"`) دالة مساعدة تُترجم قيمًا نصية مرنة `IsFlagEnabled` صراحة في ملف الإعدادات لتعطيل العلم حتى لو كان المفتاح موجودًا `dry-run=false` أيضًا بكتابة.
- يتجاهل الأسطر الفارغة، (`File.ReadAllLines`) يقرأ ملفًا نصيًا بسيطًا سطرًا سطرًا `TryLoadConfigFile` عبر صياغة `Substring` و `IndexOf('=')` والتعليقات (`#`)، ويقسم كل سطر عند أول علامة `=` باستخدام `Range: line[..separatorIndex]` و `line[(separatorIndex + 1)..]`.

7) Services/FileOrganizerService.cs (المنسق الرئيسي)

النقاط الجوهرية:

- حراسات الأمان (**Guards**) في البداية: التحقق من وجود `Source`، ثم رفض `Source == Target`، ثم رفض `Target` داخل `Source`. كل حارس يرمي استثناءً واضحًا فورًا (`... throw new`) بدل السماح للتنفيذ بالاستمرار في حالة غير آمنة — مبدأ `"Fail Fast"`.
- `NormalizeForComparison`: تحوّل أي مسار (نسبي أو بحروف مختلطة أو بفواصل زائد في النهاية) إلى صيغة موحدة قابلة للمقارنة العادلة، عبر `Path.GetFullPath` (تحويل لمسار مطلق) ثم `Path.TrimEndingDirectorySeparator` (إزالة `\` أو `/` الزائدة في النهاية).

- **IsSubPathOf**: تتحقق ما إذا كان مسار "الهدف" يبدأ بمسار "المصدر" متبوعاً بفاصل مجلد، وهو الفحص الصحيح لمعرفة "هل هذا المجلد بداخل ذلك؟" (وليس فقط **StartsWith** بسيطة، التي كانت ستُخطئ مثلاً في اعتبار **C:\SourceBackup** بداخل **C:\Source**).
- — **Directory.EnumerateFiles(options.SourceDirectory, "*", searchOption)** نستخدم **EnumerateFiles** (كسول/lazy) بدل **GetFiles** (يبني مصفوفة كاملة فوراً) لأنها أكفأ ذاكرياً مع عدد كبير من الملفات، وهي بطبيعتها لا تُعيد المجلدات إطلاقاً — وهذا بالضبط ما يحقق متطلب "تجاهل المجلدات" دون أي كود إضافي.
- **ProcessFile** هي الدالة التي تُطبّق على كل ملف على حدة: تصنيف → حساب الوجهة المرغوبة → حل التعارض → تنفيذ أو تخطيط حسب **options.DryRun** → النقاط أي استثناء متعلق بذلك الملف فقط وتحويله إلى **FileOperationResult** بحالة **Error** بدل إسقاط التطبيق بأكمله.
- — **File.Move(sourcePath, resolvedDestination, overwrite: false)** — المعامل الثالث الصريح **overwrite: false** هو خط الدفاع الأخير (بعد فحص **ConflictResolver** المسبق) ضد الكتابة فوق أي ملف، حتى في الحالة النادرة لتعارض توقيت (race condition).

8) **Logging/OrganizerLogger.cs**

- يجمع كل الأسطر المطبوعة في الذاكرة، لنُكتب لاحقاً دفعة واحدة إلى ملف السجل عبر **StringBuilder _buffer** **FlushToFile()** (أقل I/O) أكفأ من كتابة السطر تلو الآخر على القرص —
- في آن واحد، حتى لا نكرر **buffer** ونُضيف إلى الـ **Console.WriteLine** (تطبع على الطرفية **WriteLine** المنطق مرتين).
- لتنسيق **OperationStatus** enum على قيم **switch (result.Status)** نستخدم **LogOperation** تنسيق موحد يسهل قراءته — (**[ERROR]**, **[SKIPPED]**, **[MOVED]**, **[DRY-RUN]**) كل حالة بشكل مختلف إن احتجت ذلك (parsing) وتحليله لاحقاً.

9) **Program.cs** (نقطة الدخول)

- يستخدم **Top-Level Statements** (ميزة C# 9): لا حاجة لكتابة **class Program { static void Main(string[] args) { ... } }** صراحة — الملف كله هو جسم دالة **Main** ضمناً، و **args** متاحة تلقائياً. هذا يقلل الشيفرة الشكلية (boilerplate) في نقطة دخول بسيطة كهذه.
- ثوابت **const int ExitSuccess = 0**; ... توثق معنى كل **exit code** بدل أرقام سحرية (magic numbers) متناثرة في الكود.
- **CommandLineOptions.TryParse(args, out var options, out var parseError)** — نمط "Try" القياسي في .NET (**bool TryX(..., out result)**) يتجنب استخدام الاستثناءات (exceptions) لحالات "فشل متوقع" مثل معاملات ناقصة، لأن الاستثناءات أبطأ وتُفترض بها أن تكون لحالات غير متوقعة فعلاً.
- بعد بناء **service.Organize(...)**، نحسب **scanned / organized / errors** باستخدام LINQ **results.Count(r => r.Status is OperationStatus.Moved or OperationStatus.Planned).is ... or** هو نمط مطابقة (Pattern Matching) حديث في C# يُقارن قيمة واحدة بعدة احتمالات بإيجاز، بدل **r.Status == A || r.Status == B**.
- **catch (Exception ex) when (ex is DirectoryNotFoundException or InvalidOperationException or IOException)** — **Exception Filter** (**when**) يلتقط فقط أنواع الاستثناءات "المتوقعة/القابلة للإصلاح من المستخدم" (مسار خاطئ، تعارض مسارات) ويطبّع رسالة نظيفة، بينما أي

استثناء آخر غير متوقع (خطأ برمجي حقيقي) لن يُنقَط هنا فيظهر بكامل تفاصيله — وهذا مقصود: لا نريد إخفاء أخطاء برمجية حقيقية خلف رسالة عامة لطيفة.

PART 4 — الاختبارات (Testing)

لماذا xUnit؟

- المعيار الفعلي (de facto standard) لاختبار مشاريع .NET الحديثة، ويأتي بدعم مباشر من قوالب `dotnet new` ومن Visual Studio/Rider.
- بناء جملة بسيط: `[Fact]` لاختبار واحد، `[Theory]` + `[InlineData]` لتشغيل نفس الاختبار بعدة مدخلات مختلفة دون تكرار الكود — استخدمناها بكثافة في `FileClassifierTests`.
- يدعم `IDisposable` على كلاس الاختبار للتنظيف التلقائي بعد كل اختبار — استخدمناه في `FileOrganizerServiceTests` لحذف المجلد المؤقت بعد كل اختبار تلقائيًا.

توزيع الاختبارات على الملفات الثلاثة

(اختبارات وحدة خالصة، بلا لمس للقرص إطلاقًا) `FileClassifierTests.cs`:

- تصنيف الصور/المستندات/الأرشيف/الصوت/الفيديو (متطلبات 1-5 من قائمتك).
- امتداد غير معروف → `Others` (متطلب 6).
- ملف بلا امتداد → `Others` (متطلب 7).
- حساسية الأحرف (`.jpg` = `JPG`) (متطلب 17).
- اسم بنقاط متعددة (متطلب 16).

(وهي `Func<string, bool>` اختبارات وحدة خالصة، عبر حقن) `ConflictResolverTests.cs`:

- لا تعارض → يُعاد الاسم الأصلي.
- تعارض واحد → `1_`.
- تعارضات متعددة → `1_`, `2_` مشغولة، فيُختار `3_` (متطلبات 11-12).
- اسم بنقاط متعددة مع تعارض (متطلب 16 مُطبَّق هنا أيضًا).

(اختبارات تكامل حقيقية، لكن حصراً داخل مجلد مؤقت) `FileOrganizerServiceTests.cs`:

- مجلد مصدر غير موجود → استثناء (متطلب 13).
- مجلد مصدر فارغ → لا نتائج (متطلب 18).
- إنشاء مجلدات الوجهة ونقل الملفات فعليًا (متطلب 9).
- عدة ملفات بفئات مختلفة معًا (متطلب 19).
- Dry-Run لا يحرك أي ملف ولا يُنشئ حتى مجلد الهدف (متطلب 10).
- ملف موجود مسبقاً في الوجهة لا يُكتب فوقه أبداً، ويُعاد تسميته (متطلبات 8+15).
- ثلاثة ملفات بنفس الاسم → تسلسل حتمي `1_`, `2_` (متطلب 12).
- اسم بنقاط متعددة أثناء تعارض حقيقي على القرص (متطلب 16).
- `Source == Target` مرفوض (متطلب 14).
- `Target` داخل `Source` مرفوض (متطلب 15 من قائمة الحالات الحديثة في طلبك).
- امتداد غير معروف/بلا امتداد فعليًا على القرص → `Others` (متطلبات 4-5 من قائمة الحالات الحديثة).

كيفية عمل العزل الآمن: كل اختبار في `FileOrganizerServiceTests` يُنشئ مجلدًا مؤقتًا فريدًا عبر `Path.Combine(Path.GetTempPath(), "FileOrganizerTests_" + Guid.NewGuid())` في الباني (Constructor)، ويحذفه في `Dispose()`. هذا يعني: لا اختبار على الإطلاق يلمس ملفاتك الشخصية، وتشغيل الاختبارات آمن ١٠٠٪ على أي جهاز، بما فيها أجهزة الـ CI/CD على GitHub Actions مستقبلاً.

الحالات التي لم تُختبر آلياً (وتم توضيح السبب بدل تجاهلها بصمت): "الملف مقفل من برنامج آخر" و "مسار طويل جداً يتجاوز حدود Windows" غير قابلة للمحاكاة بشكل موثوق ومحمول (portable) داخل اختبار وحدة عادي دون التلاعب بصلاحيات نظام التشغيل فعلياً؛ معالجتهما موجودة في الكود (`try/catch` حول `File.Move`) لكنها موثقة كـ "معالجة موجودة، غير مُختبرة آلياً" بدل ادعاء اختبارها.

PART 5 — README

ملف `README.md` الكامل (بكل الأقسام المطلوبة: Overview, Features, Technology Stack, Requirements, Installation, Project Structure, Usage, Supported File Types, Conflict Resolution, Dry Run, Logging, Error Handling, Testing, Example, Sample Output, Screenshots, Future Improvements, وجدول Internship Requirements Checklist) جاهز فعلياً في جذر `Task_01_File_Organizer/README.md` ضمن هذا التسليم — راجعه مباشرة، فهو التوثيق الرسمي المرافق للمستودع وليس نسخة منفصلة عن هذا الدليل التعليمي.

PART 6 — بنية GitHub

المستودع الرئيسي: `SD_AVIP_2026_byte`. المشروع الحالي يعيش بالكامل داخل:

```
SD_AVIP_2026_byte/  
└─ Task_01_File_Organizer/      هنا فقط (src, tests, sln) كل ملف <-
```

ملف الحل (`FileOrganizer.sln`) و `gitignore` يعيشان في جذر `/Task_01_File_Organizer` (وليس في جذر المستودع بأكمله)، لأن كل Task مستقبلي (`Task_02`, `Task_03`, ...) سيكون له حلّه الخاص ومكدّسه التقني الخاص (بعضها بايثون، بعضها ++C)، فوضع `sln`. واحد في الجذر العام لن يكون منطقيًا. لا تُنشئ مجلدات المهام الأخرى الآن — هذا مطابق تمامًا لتعليمات طلبك.

اسم مجلد المشروع بالضبط كما طلبت: `Task_01_File_Organizer`.

ملاحظة: صفحة SOP الرسمية في ملف AVIP تطلب اسم مستودع بصيغة `DomainShortHand_TaskNumber_TaskTitle_byte` (مثل `SD_1_FileOrganizer_byte`) إن كان كل Task في مستودع منفصل — بينما طلبك الحالي يضع كل الـ Tasks داخل مستودع واحد اسمه `SD_AVIP_2026_byte` مع مجلد فرعي لكل Task. اتّبعتنا تعليماتك الصريحة بما أنك حدّدت اسم المستودع والبنية بنفسك في هذا المحضر (PART 6)، وهذا لا يتعارض مع "تسليم Task واحد بحدّ ذاته قابل للفحص" لأن المسار الكامل `/Task_01_File_Organizer` يبقى مستقلاً وقابلًا للتقييم. إن كان توجيه المقيم (mentor) في AVIP يُفضّل مستودعاً منفصلاً لكل Task بالاسم الرسمي، فبإمكانك إنشاء مستودع باسم `SD_1_FileOrganizer_byte` ونسخ محتوى `/Task_01_File_Organizer` إليه مباشرة — لا تغيير أي شيء داخل الكود مطلوب لذلك.

PART 7 — Git و GitHub (Windows) أوامر

powershell

```
# 1) الانتقال إلى مجلد المستودع (بعد استنساخه/إنشائه محليًا)
cd C:\Path\To\SD_AVIP_2026_byte

# 2) فحص الحالة الحالية
git status

# 3) إضافة كل الملفات الجديدة/المعدلة إلى منطقة التجهيز (staging area)
git add .

# 4) مراجعة التغييرات المُجهّزة قبل الالتزام بها (اختياري لكن موصى به)
git diff --cached

# 5) برسالة احترافية (commit) إنشاء الالتزام
git commit -m "feat(task-01): implement .NET file organizer"

# 6) رفع الالتزام إلى GitHub
git push origin main
```

لماذا رسالة الالتزام هذه جيدة؟

Conventional Commits: تتبع اصطلاح `feat(task-01): implement .NET file organizer`

- (`docs`) أو توثيق (`fix` ميزة جديدة، وليس إصلاح خطأ) نوع التغيير: `feat`.
- Tasks يوضح فوراً أي جزء من المستودع تأثر، مفيد جداً في مستودع سيحوي عدة — (scope) النطاق: `(task-01)` لاحقاً.
- لفهم ما جرى `diff` وواضح المعنى دون الحاجة لفتح الـ (`imperative`) الوصف قصير، بصيغة الفعل الأمر.
- التي لا تفيد أحداً بعد ستة أشهر عند مراجعة السجل `"fixed stuff"` أو `"update"` تجنّبنا رسائل غامضة مثل (`git log`).

ماذا ينتمي إلى Git، وماذا لا؟

لا ينتمي إلى Git

ينتمي إلى Git

مجلدات `/bin` و `/obj` (ناتج بناء يُعاد توليده تلقائياً من الكود المصدري)

كل ملفات `.sln`، `.csproj`، `cs`

ملفات إعدادات محلية تحتوي مسارات شخصية
(مثلاً `organizer.local.config`)

`README.md`، `.gitignore`

أي بيانات اعتماد (credentials) أو مفاتيح API

(سجل `examples/sample_run.log`)

(توضيحي عام)

سجلات تشغيل حقيقية تكشف مسارات جهازك الشخصي

لقطات الشاشة في `/screenshots`

لماذا لا يجب رفع `/bin` و `/obj` ؟

- يُعاد توليدهما بالكامل من الكود المصدري بأمر `dotnet build` — رفعهما يُضخّم حجم المستودع بلا فائدة.

- يحتويان مسارات مطلقة ومعلومات خاصة بجهاز البناء (قد تختلف بين جهازك وجهاز أي شخص آخر يستنسخ المستودع)، ما قد يُسبب تعارضات (merge conflicts) لا معنى لها.
- ملف `gitignore` المُرْفَق يستثنيهما صراحة بالفعل.

لماذا لا يجب أن تظهر مسارات شخصية في التوثيق؟

لأن `README.md` وأمثلة الاستخدام تُقرأ من قِبل أي شخص يزور المستودع (بما فيهم مسؤولو توظيف يفحصون بورتفوليو GitHub الخاص بك) — استخدام مسارات عامة (`C:\Users\Me\Downloads`) بدل اسم مستخدمك الحقيقي يحافظ على الاحترافية ويحمي خصوصيتك.

لماذا لا يجب رفع معلومات حساسة أبدًا؟

أي مفتاح/كلمة مرور تُرفع إلى Git تبقى في تاريخ المستودع (history) للأبد حتى لو حذفناها لاحقًا في التزام جديد — إنها فعليًا تتطلب إعادة كتابة التاريخ بالكامل (`git filter-repo`) أو ما شابه، وهي عملية خطيرة ومزعجة. القاعدة الذهبية: لا شيء حسّاس يدخل Git من الأساس.

PART 8 (أوامر جاهزة للنسخ) Windows إعداد وتشغيل على —

powershell

```

# 1) المثبت .NET. التحقق من إصدار
dotnet --version

# 2) المثبتة على جهازك SDKs عرض كل الـ
dotnet --list-sdks
# في القائمة، نزلهُ من NET 10. إن لم يظهر
https://dotnet.microsoft.com/download

# 3) .إن كنت تبدأ من الملفات المرفقة في هذا التسليم، لا حاجة لإنشاء مشروع جديد
# Task_01_File_Organizer: فقط تأكد من أنك داخل مجلد
cd C:\Path\To\SD_AVIP_2026_byte\Task_01_File_Organizer

# 4) (وما يتبعها xUnit) اللازمة لمشروع الاختبارات (NuGet) استعادة الحزم
dotnet restore

# 5) بناء الحل بأكمله (المشروع الرئيسي + مشروع الاختبارات)
dotnet build

# 6) تشغيل البرنامج في الوضع العادي
dotnet run --project src\FileOrganizer -- --source "C:\Users\Me\Downloads" --t

# 7) (معاينة بلا أي تنفيذ فعلي) Dry-Run تشغيل البرنامج في وضع
dotnet run --project src\FileOrganizer -- --source "C:\Users\Me\Downloads" --t

# 8) تشغيل كل الاختبارات الآلية
dotnet test

# 9) (Release build) بناء نسخة إصدار جاهزة للنشر
dotnet publish src\FileOrganizer -c Release

```

شرح كل أمر

1. `dotnet --version` — النشط حاليًا (المستخدم افتراضيًا في المجلد الحالي) SDK يطبع رقم إصدار الـ.
2. `dotnet --list-sdks` — المثبتة فعليًا على جهازك، مفيد للتأكد من توفر NET SDK. يسرد كل إصدارات . قبل المحاولة 10.
3. في المجلد الحالي `.sln` / `.csproj` التالية تبحث عن ملفات `dotnet` الانتقال للمجلد الصحيح ضروري لأن كل أوامر `--project` أو تحتاج تحديدها صراحةً).
4. أي حزمة مذكورة في NuGet ويُنزل من `.csproj` يقرأ كل ملفات `dotnet restore` — `xunit`، `xunit.runner.visualstudio`، `<PackageReference>` (هنا: غير موجودة محليًا بعد `Microsoft.NET.Test.Sdk` (لمشروع الاختبارات فقط — المشروع الرئيسي لا يحتاج أي حزمة خارجية).
5. داخل `(.dll / .exe)` إلى ملفات تنفيذية/مكتبات `.cs` كل ملفات `(compile)` يترجم `dotnet build` — `dotnet run --project src\FileOrganizer -- <args>` لكل مشروع، ويُبلغ عن أي خطأ ترجمة. 7-6 `bin/Debug/net10.0/` يبني (إن لزم) ثم يُشغل المشروع المحدد مباشرة. كل ما يأتي بعد `--` يُمرّر — `Program.cs` داخل `args` كما هو إلى نفسه ومعاملات برنامجك `dotnet` الـ `--` هنا فاصل بين معاملات —

6. `dotnet test` (يُشغَّل `.csproj` في `IsTestProject=true` عبر) يكتشف مشروع الاختبارات تلقائيًا — `[Fact]` أو `[Theory]` كل دالة موسومة بـ. ويطبّع تقريرًا (كم اختبار نجح/فشل)، `[Theory]` أو `[Fact]` كل دالة موسومة بـ.
7. `dotnet publish -c Release` (بمعنى `Debug` بعكس) يبني نسخة مُحسَّنة للأداء — `bin/Release/net10.0/publish/` الفعلي، وتخرج إلى.

PART 10 — تدفق التنفيذ الكامل (مثال حقيقي)

نفترض:

```
SOURCE/
├─ photo.jpg
├─ report.pdf
├─ song.mp3
├─ movie.mp4
├─ backup.zip
├─ unknown.xyz
└─ report.pdf
```

نفترض أن هذا اسم ملف ثانٍ بنفس الاسم كان موجودًا مسبقًا في - > `report.pdf` وليس تكرارًا حرفيًا في نفس المجلد، لأن نظام الملفات لا يسمح أصلاً بملفين بنفس الاسم في نفس `TARGET/Documents` (المجلد)

الأمر المُشغَّل:

```
powershell

dotnet run --project src\FileOrganizer -- --source "SOURCE" --target "TARGET"
```

1. `Program.cs` (جسم) وينفذ نقطة الدخول `FileOrganizer.dll` يحمل runtime يبدأ التطبيق: الـ `.NET`.
2. `Program.cs` يبدأ التنفيذ من أول سطر في `(Top-Level Statements` ضمناً عبر) `Program.Main`.
3. `--source` يقرأ `CommandLineOptions.TryParse(args, ...)`: تحليل معاملات سطر الأوامر `DryRun = false` فيضبط `--dry-run` لا يجد، `SOURCE --target TARGET`.
4. وبقية الإعدادات (`Path.GetFullPath` بعد) كائن واحد يحمل المسارين المُطلَقين: `OrganizerOptions` إنشاء.
5. نستمر — `true` تُعيد `Directory.Exists(options.SourceDirectory)`: التحقق من مجلد المصدر.
6. يُنشئ — `Directory.CreateDirectory(options.TargetDirectory)`: تجهيز مجلد الهدف `(idempotent)` لا يفعل شيئاً إن كان موجوداً بالفعل، لأن هذه الدالة "أمنة عند التكرار" (إن لم يكن موجوداً `TARGET/`).
7. تُعيد 6 `Directory.EnumerateFiles(SOURCE, "*", TopDirectoryOnly)`: تعداد الملفات مسارات (المجلدات، إن وُجدت، تُتجاهل تلقائياً لأن هذه الدالة لا تُعيدّها أصلاً).
8. لا يظهر في القائمة إطلاقاً `SOURCE` أي مجلد فرعي داخل: (نتيجة تلقائية من الخطوة 7).
9. `Path.GetExtension`: استخراج الامتداد لكل ملف `(.jpg, .pdf, .mp3, .mp4, .zip, .xyz)`.
10. `FileClassifier.Classify` تُعيد `Images, Documents, Audio, Video, Archives, Others`: تصنيف كل ملف بالترتيب.
11. `category.ToString()` اختيار فئة الوجهة: كل نتيجة تصنيف تُحوّل مباشرة لاسم مجلد عبر.

12. تُنشئ `Directory.CreateDirectory(categoryFolder)`: إنشاء مجلدات الوجهة
عند أول ملف من كل فئة ... `TARGET\Images` , `TARGET\Documents`.
13. لكل ملف `Path.Combine(categoryFolder, fileName)`: حساب اسم الملف الوجهة.
14. موجودًا مسبقًا `TARGET\Documents\report.pdf` إن كان ، `report.pdf` فحص تعارض الاسم: بالنسبة لـ
(من تشغيل سابقة مثلاً)، يُكتشف التعارض هنا.
15. `report_1.pdf` يُعيد `ConflictResolver`: توليد اسم حتمي إن لزم.
16. `File.Move(sourcePath, resolvedDestination, overwrite: false)`: نقل الملف فعليًا.
17. `OrganizerLogger.LogOperation` يطبع سطر `[MOVED] report.pdf -> Documents\report_1.pdf (renamed to avoid conflict)`: التسجيل.
18. `Files scanned:` يطبع `logger.LogSummary(6, 6, 0)`، الملخص النهائي: بعد معالجة كل الملفات الستة
`6` , `Files organized: 6` , `Errors: 0`.
19. `errors == 0` لأن (0) `ExitSuccess` يُرجع `Program.cs`: إنهاء العملية.

الشجرة النهائية:

```
TARGET/
├─ Images/
│   └─ photo.jpg
├─ Documents/
│   └─ report_1.pdf  (إن لم يوجد تعارض فعلي report.pdf أو)
├─ Audio/
│   └─ song.mp3
├─ Video/
│   └─ movie.mp4
├─ Archives/
│   └─ backup.zip
└─ Others/
    └─ unknown.xyz
```

PART 11 — Dry-Run تدفق وضع

نفس المثال، لكن بالأمر:

```
powershell

dotnet run --project src\FileOrganizer -- --source "SOURCE" --target "TARGET"
```

الفرق الجوهرى يبدأ من الخطوة 6 أعلاه:

- لا يُستدعى `Directory.CreateDirectory` للهدف على الإطلاق — الكود في `FileOrganizerService.Organize` يحيط هذا الاستدعاء بشرط `if (!options.DryRun)`.
- الفحص (خطوات 7-15) يحدث بالضبط كما في الوضع العادي — التصنيف وحل التعارض يعتمدان فقط على القراءة (`File.Exists`)، وليس على أي تعديل.

- عند الوصول لخطوة "التنفيذ"، الكود في `ProcessFile` يتحقق من `options.DryRun` ويُعيد نتيجة بحالة `Planned` دون استدعاء `Directory.CreateDirectory(categoryFolder)` ولا `File.Move` إطلاقًا.
- السجل يطبع `[DRY-RUN] photo.jpg -> Images\photo.jpg [MOVED]` بدل `[MOVED]`.
- في نهاية التشغيل: مجلد `SOURCE` يبقى تمامًا كما كان، بنفس ٦ ملفات في نفس مكانها، ومجلد `TARGET` غير موجود أصلاً إن لم يكن موجودًا من قبل.
مثال مخرجات واقعية:

```
=====
FILE ORGANIZER
=====
Mode: DRY RUN
Source: C:\SOURCE
Target: C:\TARGET

[DRY-RUN] photo.jpg -> Images\photo.jpg
[DRY-RUN] report.pdf -> Documents\report.pdf
[DRY-RUN] song.mp3 -> Audio\song.mp3
[DRY-RUN] movie.mp4 -> Video\movie.mp4
[DRY-RUN] backup.zip -> Archives\backup.zip
[DRY-RUN] unknown.xyz -> Others\unknown.xyz

Summary:
Files scanned: 6
Files organized: 6
Errors: 0
=====
```

PART 12 — الحالات الحدية (Edge Cases)

#	الحالة	السلوك المُختار
1	مجلد المصدر غير موجود	رمي <code>DirectoryNotFoundException</code> برسالة واضحة، توقف فوري
2	مجلد الهدف غير موجود	يُنشأ تلقائيًا (وضع عادي فقط)
3	مجلد المصدر فارغ	لا نتائج، لا أخطاء، ملخص يعرض أصفارًا
4	ملف بلا امتداد يُصنّف <code>Others</code>	<code>fileClassifier.Classify</code> اختبار <code>Try_ReturnsNoResults</code>

#	الحالة	السلوك المُختار
5	ملف بامتداد غير معروف	يُصنّف Others نفس ما سبق + اختبار turnsOthers
6	ملفان بنفس الاسم	الثاني يُعاد تسميته 1_ + اختبار ConflictResolver
7	ثلاثة ملفات أو أكثر بنفس الاسم	تسلسل حتمي 1_ , 2_ , 3_ ... اختبار eeFilesWithSameName
8	اسم بنقاط متعددة	اللاحقة تُضاف قبل الامتداد الأخير فقط ConflictResolver (استخدام + اختبارات مخصصة
9	امتداد بحروف كبيرة	يُعامل كنفس الامتداد الصغير ()ToLowerInvariant. في
10	تعدّر الوصول ملف	يُسجّل كـ Error لهذا الملف فقط، التشغيل يستمر try/catch في rocessFile
11	ملف مقفل من برنامج آخر	نفس معالجة #10 (يظهر عادة كـ IOException) نفس ما سبق (غير قابل للاختبار الآلي المحمولا
12	تعدّر إنشاء مجلد الوجهة	يُنقَط، يُسجّل الملف كخطأ IOException / UnauthorizedAccessException نفس آلية try/catch
13	ملف الوجهة موجود مسبقاً	لا يُكتب فوقه أبداً، يُعاد تسمية الجديد ConflictResolver + (se)
14	Source == Target	مرفوض فوراً قبل أي عملية حارس صريح في Organize + اختبار
15	Target داخل Source	مرفوض فوراً (لتفادي إعادة معالجة الملفات المنقولة حديثاً في نفس الجولة أو جولة لاحقة) حارس IsSubPathOf + اختبار e
16	عدد كبير جداً من الملفات	Directory.EnumerateFiles كسول (lazy streaming) بدل تحميل كل شيء في الذاكرة دفعة واحدة، فيبقى الأداء معقولاً تصميم الخدمة (merable<string)
17	مسارات/أسماء طويلة جداً (حدود (Windows	PathTooLongException مُدرّجة صراحة في قائمة الاستثناءات catch في ProcessFile
18	مدخلات نظام ملفات خاصة	تُعامل كملفات عادية من منظور Directory.EnumerateFiles ؛ لم تُضف معالجة خاصة موثقة كقرار واعٍ، غير مُنفذة

#	الحالة	السلوك المُختار
(رمزية، إلخ)	تنزيلات "العادي" — هذه نقطة تحسين مستقبلي محتملة إن لزم	إضافية لأنها خارج نطاق المتطلبات الرسمية ونادرة الحدوث في سيناريو "تنظيم
(روابط		

PART 13 (Security & Safety Review) مراجعة الأمان والسلامة —

- لا كتابة فوق أي ملف: مضمونة بطبقتين مستقلتين — الفحص المسبق في `ConflictResolver` (طبقة القرار)، والمعامل الصريح `overwrite: false` في `File.Move` (طبقة التنفيذ، خط دفاع أخير ضد أي تعارض توقيت نادر).
- **Dry-Run** آمن فعليًا وليس شكليًا فقط: تتبعت كل استدعاء لأي API يُغيّر حالة القرص (`File.Move`, `Directory.CreateDirectory`) وتأكدت أن كلاهما محاط بشرط `if (!options.DryRun)` في المسار التنفيذي، وليس مجرد رسالة نصية مختلفة فقط.
- معالجة مسارات المستخدم بأمان: كل مسار يُمرّ عبر `Path.GetFullPath` (تطبيع/normalization) قبل استخدامه في أي مقارنة أو عملية، بدل الوثوق بالنص الخام كما أُدخل.
- لا تُنفذ أي أوامر **Shell**: التطبيق لا يستدعي `Process.Start` ولا أي واجهة تنفيذ أوامر خارجية إطلاقًا — كل التعامل عبر `System.IO` المُدار (managed) مباشرة.
- لا تنفيذ كود عشوائي: لا `eval`، لا تحميل تجميعات (assemblies) ديناميكيًا، لا reflection غير آمن.
- لا مسارات شخصية مُضمّنة في الكود: كل الأمثلة في الكود والتوثيق تستخدم مسارات عامة (`...\\C:\\Users\\Me`) أو أوسمة مسار (`SOURCE`, `TARGET`) بدل مسار حقيقي على أي جهاز.
- الأخطاء لا تُسبب سلوكًا مدمرًا: أي استثناء أثناء معالجة ملف واحد يُحوّل لنتيجة `Error` بدلًا من ترك الملف في حالة غير معروفة أو إسقاط التطبيق بأكمله في منتصف عملية نقل جماعية.
- التطبيق لا يُعيد معالجة ناتجه الخاص: الحارسان `Source == Target` و `"Target inside Source"` يمنعان تحديدًا سيناريو أن يقوم البرنامج، عبر `recursive--` أو تشغيل متكرر، بمعالجة الملفات التي نقلها هو نفسه للتو.

PART 14 (Senior Code Review) مراجعة كود احترافية —

الملاحظات	المعيار	التقييم (10/10)
فصل واضح بين القرار والتنفيذ؛ نقطة خصم بسيطة لعدم وجود واجهات (interfaces) قابلة للاستبدال بالكامل عبر DI حقيقي — قرار مقصود للبساطة، مذكور صراحة.	البنية المعمارية (Architecture)	9
كل متطلب رسمي مُغطى ومُختبر منطقيًا؛ لم يُنفذ فعليًا (انظر PART 15) فالتقييم "منطقي" لا "موكّد بالتنفيذ".	الصحة الوظيفية (Correctness)	9
<code>PascalCase</code> لأنواع والدوال، <code>camelCase</code> للمتغيرات المحلية، <code>camelCase_</code> للحقول الخاصة، <code>records</code> للبيانات غير القابلة للتغيير — متسق عبر كل الملفات.	اتّباع اصطلاحات #C	9
استخدام سليم لـ <code>System.IO</code> ، أنماط nullable reference types، <code>Try*</code> مفعّلة وموظّفة فعليًا وليست مجرد إعداد شكلي.	اتّباع اصطلاحات .NET	9
تعليقات XML على كل كلاس/دالة عامة تشرح "لماذا" وليس فقط "ماذا"؛ أسماء دوال ومتغيرات واضحة المعنى.	القابلية للقراءة (Readability)	9

قابلية الصيانة (Maintainability)	8	إضافة امتداد جديد = سطر واحد في <code>FileClassifier</code> ؛ إضافة علم CLI جديد يتطلب تعديل مكانين (switch + بناء Options) — نقطة تحسين محتملة مستقبلاً.
معالجة الأخطاء	8	شاملة على مستوى الملف الواحد والتشغيل ككل؛ لا تفريق حالياً بين أنواع فرعية أدق من IOException (قرار تبسيط واع).
سلامة نظام الملفات	10	لا كتابة فوق أي ملف بطبقتين مستقلتين، Dry-Run حقيقي 100%، حراسات صريحة ضد Source/Target المتداخلين.
السلوك عبر الأنظمة (Cross-platform)	9	لا مسارات مكتوبة يدوياً بفواصل \؛ الاعتماد الكامل على <code>Path.Combine</code> / <code>Path.DirectorySeparatorChar</code> يجعله يعمل على Linux/macOS أيضاً رغم أن المتطلب الرسمي يذكر Windows فقط.
الأداء	8	<code>EnumerateFiles</code> كسول ومناسب لأعداد كبيرة؛ لا معالجة متوازية (parallel) — غير ضرورية لحجم الاستخدام النموذجي (مجلد تنزيلات شخصي)، لكنها نقطة تحسين ممكنة لمجلدات ضخمة جداً.
تغطية الاختبارات	9	تغطي كل المتطلبات الوظيفية الرسمية وأغلب الحالات الحدية المطلوبة (٢٠ حالة مطلوبة، مُغطاة فعلياً أو مؤثقة بسبب عدم التغطية الآلية).
قابلية استخدام CLI	8	رسائل خطأ واضحة، دعم config file بديل؛ لا <code>help--</code> مخصص حالياً (نقطة تحسين مستقبلية بسيطة).
التسجيل (Logging)	9	يغطي كل ما طُلب رسمياً (البداية، المسارات، الوضع، كل عملية، الملخص) بتنسيق مقروء ومتسق.
التوثيق	9	<code>README.md</code> يغطي كل الأقسام المطلوبة بما فيها جدول التزام المتطلبات الرسمية.
جاهزية GitHub	9	<code>gitignore.</code> صحيح، بنية مجلدات نظيفة، لا مسارات شخصية.
الالتزام بمتطلبات التدريب	10	كل بند رسمي مُنفذ دون إسقاط أو تخفيف، والتمييز بين الرسمي/الاقتراحي/الاختياري واضح في <code>README.md</code> .

المشكلات التي رُصدت أثناء المراجعة الأولى، وكيف عُولجت (قبل التسليم النهائي):

- كان هناك كود ميت (غير مُستخدَم) لحساب "مسار نسبي" داخل `OrganizerLogger.LogOperation` بمنطق غير ضروري ومُعقّد بلا فائدة فعلية — تمت إزالته واستُبدل بمنطق مباشر يعتمد على `result.Category` المتوفرة أصلاً.
- كانت أعلام `--recursive` / `dry-run` القادمة من ملف الإعدادات تُفعّل بمجرد وجود المفتاح بغض النظر عن قيمته (فحتى `dry-run=false` كانت ستُفعّل الوضع الجاف خطأً) — تم إصلاحه بدالة `IsFlagEnabled` التي تفسّر القيمة الفعلية (1 / false / true / فارغ) بدل الاكتفاء بفحص وجود المفتاح.

PART 15 — التحقق من إمكانية البناء (Build Verification)

كما ذكر في بداية هذا الملف: لم أشغل `dotnet build` أو `dotnet test` فعليًا (بيئتي لا تملك .NET SDK). ما قمتُ به هو تحقق منطقي شامل، يشمل:

- ✓ مراجعة كل `using` والتأكد أنها تُطابق مساحات الأسماء المُستخدمة فعليًا (`FileOrganizer.Models`), `FileOrganizer.Services`, `FileOrganizer.Logging`, `FileOrganizer.Configuration`).

- ✓ التأكد من تطابق توقيعات الدوال بين مكان التعريف ومكان الاستدعاء (مثال: تعديل `OrganizerLogger.LogOperation` من معاملين إلى معامل واحد، وتحديث نقطة الاستدعاء الوحيدة في `FileOrganizerService` بما يتوافق).

- ✓ التأكد من تهيئة كل خاصية `required` في كل مكان يُنشأ فيه `OrganizerOptions` و `FileOperationResult`.

- ✓ التأكد من صحة استخدام (`string?`) `Nullable Reference Types` وعدم وجود مسار كود يُمرّر `null` إلى معامل غير `Nullable`.

- ✓ التأكد من تطابق أسماء المشاريع بين `FileOrganizer.sln` ومسارات ملفات `csproj` الفعلية.

- ✓ التأكد من أن مشروع الاختبارات يحمل `<ProjectReference>` صحيح المسار النسبي (`src\FileOrganizer\FileOrganizer.csproj\..\..`) بالنسبة لموقعه الفعلي.

- ✓ مراجعة كل صياغة `C#` حديثة مُستخدمة (`records`, `collection expressions` [`...`], `required`, `pattern matching` `is ... or`, `range operator` `...`) والتأكد من توفرها ضمن إصدار اللغة الذي يجلبه `net10.0` تلقائيًا (`C# 14`، وكلها متوفرة منذ إصدارات أقدم بكثير — `C# 9` إلى `12`).

ما لم أفعله ولن أدعي فعله: لم أشغل الاختبارات، لم أرَ مخرجات `dotnet test` الفعلية، ولم أنتج أي "نتيجة نجاح ١٠٠%" مُختلفة. هذه خطوة يجب أن تنفذها أنت بالأوامر في PART 8 فور استلام المشروع، وقبل الدفع إلى GitHub أو تصوير أي لقطة شاشة.

PART 16 (Internship Deliverables) تجهيز تسليمات التدريب —

1. مستودع GitHub: أنشئ مستودعًا عامًا (`public`) باسم `SD_AVIP_2026_byte`، وارفع محتوى هذا المشروع بالكامل داخل `/Task_01_File_Organizer` بالتّباع أوامر PART 7.
2. README: جاهز فعليًا (`README.md`) — لا حاجة لأي إجراء إضافي عدا استبدال `<your-username>` في رابط الاستنساخ باسم حسابك الحقيقي.
3. سجل تشغيل نموذجي: جاهز فعليًا (`examples/sample_run.log`) — لكن يُفضّل أن تستبدله (أو تُضيف إليه) بسجل ناتج فعليًا من تشغيل حقيقية على جهازك بعد التحقق من البناء، ليعكس نتيجة حقيقية وليس نصًا افتراضيًا فقط.
4. لقطة شاشة "قبل": افتح مجلد تجريبي يحتوي خليطًا من الملفات (صور، PDF، صوت...) قبل أي تشغيل، وصوره كاملاً في مستكشف الملفات (File Explorer). احفظه في `screenshots/before.png`.
5. لقطة شاشة "بعد": بعد تشغيل البرنامج فعليًا (وضع عادي)، صور نفس المجلد الهدف موضحًا المجلدات الفرعية الجديدة (`Images`, `Documents`...). احفظه في `screenshots/after.png`.

أي اللقطات إلزامية وأيها اختيارية؟

- إلزامي (مطلوب رسميًا في AVIP): لقطة "قبل" ولقطة "بعد".
- اختياري لكن موصى به بقوة: لقطة أثناء تشغيل `dry-run--` (تثبت أن وضع المعاينة يعمل فعليًا)، ولقطة لبنية المستودع على GitHub (تظهر احترافية التنظيم لأي زائر للبروفایل).

PART 17 قائمة التحقق النهائية قبل التسليم —

[] dotnet --version مثبت فعليًا وتم التحقق منه بـ (أو ما يعادله 10) .NET SDK []
[] بدون أي خطأ (dotnet build) المشروع يُبنى بنجاح []
[] (dotnet test) كل الاختبارات تنجح فعليًا []
[] الوضع العادي يعمل ويحرك الملفات بشكل صحيح []
[] يعمل ولا يحرك أي ملف فعليًا Dry-Run وضع []
[] Others + تصنيف الملفات يعمل لكل الفئات الخمس []
[] (Others) الملفات ذات الامتدادات غير المعروفة تُعالج بأمان []
[] (Others) الملفات بلا امتداد تُعالج بأمان []
[] حل التعارضات يعمل بشكل حتمي (1_, 2_, ...) []
[] لا يتم أبدًا الكتابة فوق ملف موجود مسبقًا []
[] معالجة الأخطاء تعمل (ملف واحد فاشل لا يوقف التشغيل بأكمله) []
[] مكتمل بكل الأقسام المطلوبة README []
[] سجل تشغيل نموذجي حقيقي تم إنشاؤه بعد تشغيل فعلي على جهازك []
[] لقطة شاشة "قبل" تم التقاطها []
[] تم التقاطها (اختياري لكن موصى به) Dry-Run لقطة شاشة []
[] لقطة شاشة "بعد" تم التقاطها []
[] تم تهيئته محليًا Git مستودع []
[] (SD_AVIP_2026_byte) اسم المستودع صحيح []
[] (Task_01_File_Organizer) صحيح Task اسم مجلد []
[] bin/ و obj/ مُهيأ ويستثني .gitignore []
[] احترافي تم إنشاؤه (commit) التزام []
[] (public) عام GitHub المستودع على []
[] GitHub الكود مرفوع فعليًا إلى []
[] الرسمية لهذه المهمة محققة AVIP كل متطلبات []
[] ArithMatrix جاهز للتسليم في []

Content is user-generated and unverified.

Content is user-generated and unverified.